

Projet B - Séquence 3.6 -

Script de création de la base de données

Script SQL Final pour Supabase

Voici le script prêt à être exécuté dans Supabase (via l'onglet **SQL Editor**).

```
-- =====
-- 1. Activation des extensions nécessaires
-- =====
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- =====
-- 2. Création des types ENUM
-- =====
CREATE TYPE role_utilisateur AS ENUM ('étudiant', 'professeur', 'admin');
CREATE TYPE statut_utilisateur AS ENUM ('actif', 'inactif', 'suspendu');
CREATE TYPE type_ressource AS ENUM ('Document', 'Video', 'Audio', 'Lien');
CREATE TYPE statut_tache AS ENUM ('planifiée', 'à faire', 'en cours', 'terminée');
CREATE TYPE priorite_tache AS ENUM ('basse', 'moyenne', 'haute');
CREATE TYPE type_calendrier AS ENUM ('personnel', 'cours', 'examens', 'groupe');
CREATE TYPE statut_notification AS ENUM ('non lue', 'lue', 'archivée');
CREATE TYPE type_abonnement AS ENUM ('gratuit', 'étudiant', 'professeur', 'institutionnel');
CREATE TYPE statut_plan_revision AS ENUM ('actif', 'terminé', 'annulé');
CREATE TYPE statut_session_revision AS ENUM ('planifiée', 'en cours', 'terminée');
CREATE TYPE type_question_quiz AS ENUM ('QCM', 'texte libre', 'vrai/faux');
CREATE TYPE type_donnees_analytiques AS ENUM ('temps_étude', 'score_quiz', 'connexion');

-- =====
-- 3. Tables principales
-- =====

-- Table Établissement
CREATE TABLE etablisement (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(255) NOT NULL,
  adresse TEXT,
  ville VARCHAR(100),
  pays VARCHAR(100),
  code_postal VARCHAR(20),
  logo BYTEA,
  date_creation TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

```

-- Table Utilisateur
CREATE TABLE utilisateur (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(100) NOT NULL,
  prénom VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  mot_de_passe TEXT NOT NULL,
  rôle role_utilisateur NOT NULL DEFAULT 'étudiant',
  date_inscription TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  dernière_connexion TIMESTAMP WITH TIME ZONE,
  photo_profil BYTEA,
  statut statut_utilisateur NOT NULL DEFAULT 'actif',
  id_établissement UUID REFERENCES etablissement(id) ON DELETE SET NULL
);

-- Table Cursus
CREATE TABLE cursus (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(255) NOT NULL,
  description TEXT,
  id_établissement UUID NOT NULL REFERENCES etablissement(id) ON DELETE CASCADE
);

-- Table Cours
CREATE TABLE cours (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(255) NOT NULL,
  description TEXT,
  id_professeur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  id_cursus UUID NOT NULL REFERENCES cursus(id) ON DELETE CASCADE,
  date_création TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  date_mise_à_jour TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Table Ressource
CREATE TABLE ressource (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  titre VARCHAR(255) NOT NULL,
  type type_ressource NOT NULL,
  contenu TEXT,
  id_cours UUID REFERENCES cours(id) ON DELETE CASCADE,
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  date_création TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  note_moyenne FLOAT DEFAULT 0
);

-- Table Tâche
CREATE TABLE tâche (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  titre VARCHAR(255) NOT NULL,
  description TEXT,
  date_limite TIMESTAMP WITH TIME ZONE NOT NULL,
  statut statut_tache NOT NULL DEFAULT 'planifiée',
  priorité priorite_tache NOT NULL DEFAULT 'moyenne',

```

```

    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    id_cours UUID REFERENCES cours(id) ON DELETE CASCADE
);

-- Table Groupe_Collaboratif
CREATE TABLE groupe_collaboratif (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    nom VARCHAR(255) NOT NULL,
    description TEXT,
    id_créateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    date_création TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Table Membre_Groupe
CREATE TABLE membre_groupe (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    id_groupe UUID NOT NULL REFERENCES groupe_collaboratif(id) ON DELETE CASCADE,
    date_adhésion TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    rôle VARCHAR(50) NOT NULL,
    UNIQUE (id_utilisateur, id_groupe)
);

-- Table Calendrier
CREATE TABLE calendrier (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    titre VARCHAR(255) NOT NULL,
    description TEXT,
    date_debut TIMESTAMP WITH TIME ZONE NOT NULL,
    date_fin TIMESTAMP WITH TIME ZONE NOT NULL,
    type type_calendrier NOT NULL
);

-- Table Notification
CREATE TABLE notification (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    titre VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    date_envoi TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    statut statut_notification NOT NULL DEFAULT 'non lue'
);

-- Table Abonnement
CREATE TABLE abonnement (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    type type_abonnement NOT NULL,
    date_debut TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    date_fin TIMESTAMP WITH TIME ZONE,
    statut statut_utilisateur NOT NULL DEFAULT 'actif'
);

```

```

-- Table Plan_Révision
CREATE TABLE plan_revision (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  titre VARCHAR(255) NOT NULL,
  description TEXT,
  date_debut TIMESTAMP WITH TIME ZONE NOT NULL,
  date_fin TIMESTAMP WITH TIME ZONE NOT NULL,
  statut statut_plan_revision NOT NULL DEFAULT 'actif'
);

-- Table Session_Révision
CREATE TABLE session_revision (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_plan_revision UUID NOT NULL REFERENCES plan_revision(id) ON DELETE CASCADE,
  date TIMESTAMP WITH TIME ZONE NOT NULL,
  durée INT NOT NULL,
  statut statut_session_revision NOT NULL DEFAULT 'planifiée'
);

-- Table Quiz
CREATE TABLE quiz (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  titre VARCHAR(255) NOT NULL,
  description TEXT,
  id_cours UUID REFERENCES cours(id) ON DELETE CASCADE,
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  date_creation TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Table Question_Quiz
CREATE TABLE question_quiz (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_quiz UUID NOT NULL REFERENCES quiz(id) ON DELETE CASCADE,
  contenu TEXT NOT NULL,
  type_question_quiz NOT NULL,
  points INT NOT NULL DEFAULT 1
);

-- Table Réponse_Quiz
CREATE TABLE reponse_quiz (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_question_quiz UUID NOT NULL REFERENCES question_quiz(id) ON DELETE CASCADE,
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  contenu TEXT,
  est_correcte BOOLEAN NOT NULL DEFAULT FALSE,
  date_soumission TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  UNIQUE (id_question_quiz, id_utilisateur)
);

-- Table Données_Analytiques
CREATE TABLE donnees_analytiques (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

```

```

id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
type type_donnees_analytiques NOT NULL,
valeur FLOAT NOT NULL,
date_enregistrement TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- =====
-- 4. Index pour l'optimisation
-- =====
CREATE INDEX idx_tache_utilisateur ON tache(id_utilisateur);
CREATE INDEX idx_tache_cours ON tache(id_cours);
CREATE INDEX idx_tache_statut ON tache(statut);
CREATE INDEX idx_tache_priorite ON tache(priorite);
CREATE INDEX idx_tache_date_limite ON tache(date_limite);

CREATE INDEX idx_ressource_cours ON ressource(id_cours);
CREATE INDEX idx_ressource_utilisateur ON ressource(id_utilisateur);
CREATE INDEX idx_ressource_type ON ressource(type);
CREATE INDEX idx_ressource_note_moyenne ON ressource(note_moyenne);

CREATE INDEX idx_membre_groupe_utilisateur ON membre_groupe(id_utilisateur);
CREATE INDEX idx_membre_groupe_groupe ON membre_groupe(id_groupe);

CREATE INDEX idx_notification_utilisateur ON notification(id_utilisateur);
CREATE INDEX idx_notification_statut ON notification(statut);

CREATE INDEX idx_quiz_cours ON quiz(id_cours);
CREATE INDEX idx_quiz_utilisateur ON quiz(id_utilisateur);

CREATE INDEX idx_reponse_quiz_question ON reponse_quiz(id_question_quiz);
CREATE INDEX idx_reponse_quiz_utilisateur ON reponse_quiz(id_utilisateur);

CREATE INDEX idx_calendrier_utilisateur ON calendrier(id_utilisateur);
CREATE INDEX idx_calendrier_type ON calendrier(type);

CREATE INDEX idx_plan_revision_utilisateur ON plan_revision(id_utilisateur);
CREATE INDEX idx_session_revision_plan ON session_revision(id_plan_revision);

CREATE INDEX idx_cours_professeur ON cours(id_professeur);
CREATE INDEX idx_cours_cursus ON cours(id_cursus);

CREATE INDEX idx_cursus_etablissement ON cursus(id_etablissement);

-- =====
-- 5. Trigger pour mettre à jour dernière_connexion
-- =====
CREATE OR REPLACE FUNCTION update_derniere_connexion()
RETURNS TRIGGER AS $$
BEGIN
    NEW.derniere_connexion = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trigger_update_derniere_connexion
BEFORE UPDATE ON utilisateur
FOR EACH ROW
EXECUTE FUNCTION update_derniere_connexion();

-- =====
-- 6. Fonction pour calculer note_moyenne dans Ressource
-- =====
-- Table pour stocker les notes des ressources (si nécessaire)
CREATE TABLE note_ressource (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_ressource UUID NOT NULL REFERENCES ressource(id) ON DELETE CASCADE,
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  note FLOAT NOT NULL CHECK (note BETWEEN 0 AND 5),
  date_notation TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  UNIQUE (id_ressource, id_utilisateur)
);

-- Fonction pour recalculer la note_moyenne
CREATE OR REPLACE FUNCTION recalculer_note_moyenne()
RETURNS TRIGGER AS $$
DECLARE
  moyenne FLOAT;
BEGIN
  SELECT AVG(note) INTO moyenne
  FROM note_ressource
  WHERE id_ressource = COALESCE(NEW.id_ressource, OLD.id_ressource);

  IF moyenne IS NULL THEN
    moyenne = 0;
  END IF;

  UPDATE ressource
  SET note_moyenne = moyenne
  WHERE id = COALESCE(NEW.id_ressource, OLD.id_ressource);

  RETURN NULL;
END;
$$ LANGUAGE plpgsql;

-- Trigger pour recalculer la note_moyenne après insertion/mise à jour/suppression
CREATE TRIGGER trigger_recalculer_note_moyenne_insert
AFTER INSERT ON note_ressource
FOR EACH ROW
EXECUTE FUNCTION recalculer_note_moyenne();

CREATE TRIGGER trigger_recalculer_note_moyenne_update
AFTER UPDATE ON note_ressource
FOR EACH ROW
EXECUTE FUNCTION recalculer_note_moyenne();

CREATE TRIGGER trigger_recalculer_note_moyenne_delete
AFTER DELETE ON note_ressource
FOR EACH ROW

```

```

EXECUTE FUNCTION recalculer_note_moyenne();

-- =====
-- 7. Règles de sécurité (RLS) de base
-- =====
-- Activer RLS sur toutes les tables
ALTER TABLE utilisateur ENABLE ROW LEVEL SECURITY;
ALTER TABLE etablissement ENABLE ROW LEVEL SECURITY;
ALTER TABLE cursus ENABLE ROW LEVEL SECURITY;
ALTER TABLE cours ENABLE ROW LEVEL SECURITY;
ALTER TABLE ressource ENABLE ROW LEVEL SECURITY;
ALTER TABLE tâche ENABLE ROW LEVEL SECURITY;
ALTER TABLE groupe_collaboratif ENABLE ROW LEVEL SECURITY;
ALTER TABLE membre_groupe ENABLE ROW LEVEL SECURITY;
ALTER TABLE calendrier ENABLE ROW LEVEL SECURITY;
ALTER TABLE notification ENABLE ROW LEVEL SECURITY;
ALTER TABLE abonnement ENABLE ROW LEVEL SECURITY;
ALTER TABLE plan_revision ENABLE ROW LEVEL SECURITY;
ALTER TABLE session_revision ENABLE ROW LEVEL SECURITY;
ALTER TABLE quiz ENABLE ROW LEVEL SECURITY;
ALTER TABLE question_quiz ENABLE ROW LEVEL SECURITY;
ALTER TABLE reponse_quiz ENABLE ROW LEVEL SECURITY;
ALTER TABLE donnees_analytiques ENABLE ROW LEVEL SECURITY;
ALTER TABLE note_ressource ENABLE ROW LEVEL SECURITY;

-- Exemple de politiques RLS pour la table utilisateur
CREATE POLICY "Les utilisateurs peuvent voir leur propre profil"
ON utilisateur
FOR SELECT
USING (auth.uid() = id::TEXT);

CREATE POLICY "Les utilisateurs peuvent mettre à jour leur propre profil"
ON utilisateur
FOR UPDATE
USING (auth.uid() = id::TEXT);

-- Exemple pour la table ressource
CREATE POLICY "Les utilisateurs peuvent voir les ressources des cours auxquels ils sont inscrits"
ON ressource
FOR SELECT
USING (
    EXISTS (
        SELECT 1 FROM cours c
        JOIN membre_groupe mg ON c.id_professeur = mg.id_utilisateur
        WHERE c.id = ressource.id_cours AND mg.id_utilisateur = auth.uid()::UUID
    )
    OR
    EXISTS (
        SELECT 1 FROM cours c
        JOIN tâche t ON c.id = t.id_cours
        WHERE c.id = ressource.id_cours AND t.id_utilisateur = auth.uid()::UUID
    )
    OR

```

```

    ressource.id_utilisateur = auth.uid()::UUID
);

-- Exemple pour la table tâche
CREATE POLICY "Les utilisateurs peuvent voir leurs propres tâches"
ON tâche
FOR SELECT
USING (id_utilisateur = auth.uid()::UUID);

CREATE POLICY "Les utilisateurs peuvent créer leurs propres tâches"
ON tâche
FOR INSERT
WITH CHECK (id_utilisateur = auth.uid()::UUID);

-- Exemple pour la table groupe_collaboratif
CREATE POLICY "Les utilisateurs peuvent voir les groupes dont ils sont membres"
ON groupe_collaboratif
FOR SELECT
USING (
    id_créateur = auth.uid()::UUID
    OR
    EXISTS (
        SELECT 1 FROM membre_groupe
        WHERE id_groupe = groupe_collaboratif.id AND id_utilisateur = auth.uid()::UUID
    )
);

-- Exemple pour la table notification
CREATE POLICY "Les utilisateurs peuvent voir leurs propres notifications"
ON notification
FOR SELECT
USING (id_utilisateur = auth.uid()::UUID);

```

Résumé des fonctionnalités implémentées

Éléments	Détails
Types ENUM	Tous les enum sont créés selon vos spécifications.
Clés étrangères	ON DELETE CASCADE pour les suppressions en cascade.
Contraintes UNIQUE	Ajoutées pour Membre_Groupe et Réponse_Quiz .
Index	Optimisation des requêtes sur les champs fréquemment utilisés.
Trigger dernière_conne	Mise à jour automatique à chaque modification de l'utilisateur.
Fonction note_moyenne	Recalcule la moyenne des notes pour une ressource (via la table note_ressource).

RLS (Row-Level Security) Politiques de base pour restreindre l'accès aux données par utilisateur